FINAL PROJECT SPECIFICATION

FixIt

Smart Maintenance & Incident Reporting Platform with AI Triage

Maintenance requests get lost in WhatsApp chats. FixIt makes every request trackable, resolves simple problems instantly with AI guidance, and gives management the data to decide where to invest.

Course	COP 3851 – Web Development II (Section 915720)
Term	Fall 2026
Professor	Mr. Felix Urrutia
Student	Yaderling Garcia
Presentation	December 7, 2026
Stack	React (Vite) · Node.js · Express · PostgreSQL + Prisma · MongoDB + Mongoose · Socket.io · JWT · Vitest + Supertest · Docker · OpenAPI/Swagger

Contents

1. Executive Summary 3
2. Problem Statement 3
3. Objectives & Scope 3
4. Users, Roles & Permissions 4
5. Functional Requirements 6
6. Non-Functional Requirements 7
7. System Architecture 7
8. Data Model 9
9. REST API Design 10
10. Middleware Stack 12
11. Authentication, Authorization & Security 13
12. AI Smart Triage 14
13. Real-Time Notifications (Socket.io) 15
14. Front-End (React SPA) 16
15. Testing Strategy 16
16. Configuration, Logging, Deployment & Documentation 17
17. Project Structure 18
18. Syllabus Coverage Matrix 19
19. Development Timeline 20
20. Presentation & Demo Plan (≈ 10 minutes) 20
21. Deliverables Checklist 20
22. Risks & Mitigations 21
23. Future Work 21

1. Executive Summary

FixIt is a single-page web application backed by a RESTful Node.js/Express API that lets students, tenants or employees report maintenance problems (broken AC, leaks, dead outlets, broken projectors) and track them until they are fixed. Before a ticket is created, an **AI Smart Triage** step analyzes the photo and description, auto-fills the category and priority, and, when it is safe, suggests quick-fix steps so the user can solve simple problems on their own. Problems that need a professional become tickets that admins assign to technicians, with **real-time status updates** for everyone involved.

The project applies the eight Course Learning Objectives of COP 3851: a front-end framework, server-side JavaScript, route processing, middleware, RESTful APIs, authentication and authorization, state and persistence, and relational + non-relational databases. It also applies the testing, real-time, deployment and documentation topics from Weeks 10–14.

2. Problem Statement

2.1 The real-life problem

In universities, apartment buildings and offices, maintenance problems are usually reported through WhatsApp messages, phone calls, or verbally to whoever is nearby. This causes four recurring issues:

- **Lost requests:** messages get buried in chats; nobody owns the problem.
- **No visibility:** the person who reported it never knows if someone is working on it.
- **No accountability:** management cannot see who is handling what or how long it takes.
- **No data:** there is no history to show which buildings or categories fail most, so budget decisions are guesses.
- **Wasted technician time:** many reports are simple issues (wrong projector input, tripped breaker reset by staff, Wi-Fi reconnect) that the user could solve in a minute with guidance.

2.2 The solution

FixIt turns each report into a structured, trackable ticket with a photo, location, category and priority; uses AI to resolve simple cases instantly and pre-classify the rest; notifies reporters and technicians live; and gives admins a dashboard with resolution times and recurring problem areas.

2.3 Target users & context

Primary context: a university campus (classrooms, labs, dorms, offices). The same system works without changes for apartment complexes or office buildings, because locations and categories are managed by the admin.

3. Objectives & Scope

3.1 Project objectives

- Reduce lost maintenance requests to zero by giving every report a ticket and an owner.
- Notify reporters in real time on every status change.
- Resolve a share of simple issues without a technician through AI quick-fix guidance.

- Provide admins with metrics: average resolution time, tickets by category/location, AI self-resolution rate.
- Demonstrate the course concepts in a secure, tested, documented and containerized application.

3.2 Scope

Core (MVP – must have)	Extras (if time allows)
• Registration / login with JWT (httpOnly cookie) • Role-based access: Reporter, Technician, Admin • Ticket CRUD with photo upload • AI Smart Triage (category, priority, quick fix, hazard detection) • Assignment of tickets to technicians • Status workflow with history timeline • Filtering, sorting and pagination • Real-time notifications (Socket.io) • Validation, centralized error handling, security hardening • Integration tests for protected endpoints • Swagger/OpenAPI documentation	• Admin dashboard charts (aggregation pipelines) • CSV export of tickets (streams) • Reporter rating of the fix (1–5 stars) • SLA alert: ticket open > 48 h • Multi-stage Docker build + cloud deployment • Structured logging (Pino) + health checks • Dark mode / mobile-first polish

3.3 Out of scope

Payments, inventory of spare parts, native mobile apps, SMS/e-mail delivery (in-app notifications only), multi-organization tenancy.

4. Users, Roles & Permissions

Role	Who	Main goals
Reporter	Student, tenant, employee	Report a problem quickly, try the AI quick fix, follow the ticket status, rate the fix.
Technician	Maintenance staff	See assigned tickets, update status, add notes and resolution details.
Admin	Facilities manager	Assign tickets, manage users/categories/locations, view dashboard, export reports.

4.1 Permission matrix (RBAC)

Action	Reporter	Technician	Admin
Create ticket / run AI triage	Yes	Yes	Yes
View tickets	Own only	Assigned only	All
Edit ticket description/photo	Own, while Open	No	Yes
Change status	Cancel own (Open only)	Assigned: In Progress / Resolved	Any
Assign / reassign technician	No	No	Yes
Comment on ticket	Own	Assigned	Any
Rate resolution	Own, after Resolved	No	No
Manage categories, locations, users	No	No	Yes
Dashboard & CSV export	No	Own stats	Yes

Authorization is enforced **on the server** by the `authorize(...roles)` guard plus ownership checks in the service layer (e.g., a technician can only update tickets where `assigneeId === req.user.id`). The React UI hides actions the user cannot perform, but the UI is never the security boundary.

5. Functional Requirements

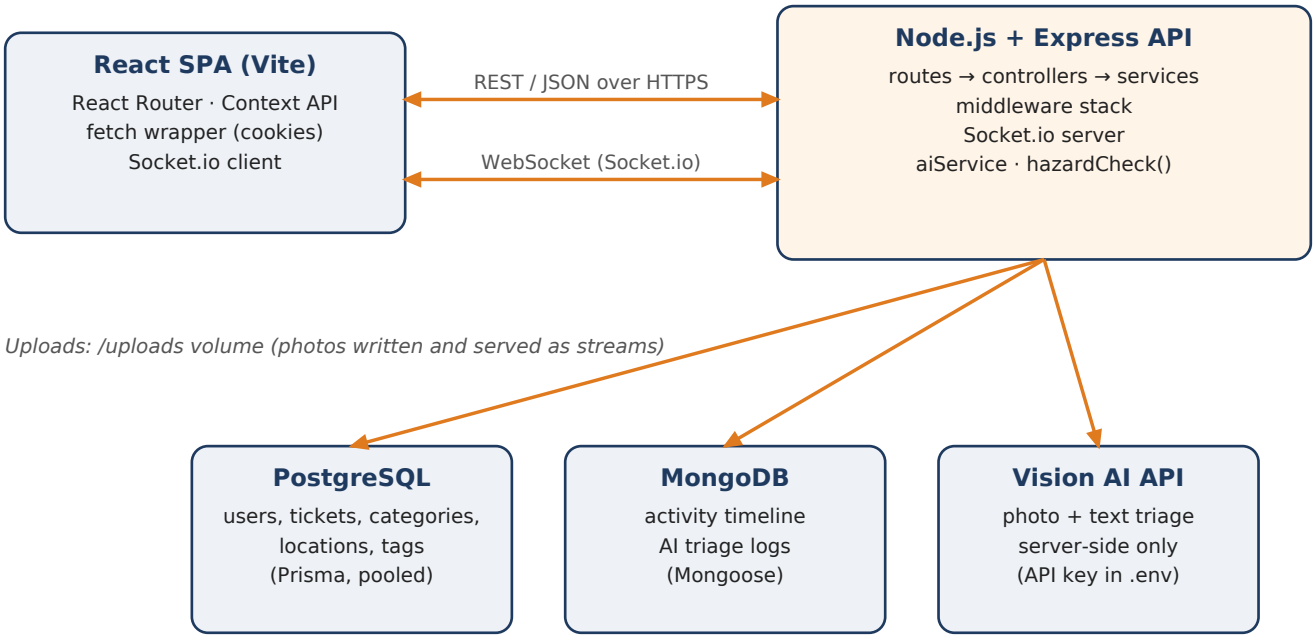
ID	Requirement	Role
FR-01	Users can register (default role Reporter), log in and log out. Passwords are hashed with bcrypt.	All
FR-02	Admin can create Technician/Admin accounts and deactivate users.	Admin
FR-03	Reporter uploads a photo (JPG/PNG/WebP, max 5 MB) and a description and receives an AI triage result.	Reporter
FR-04	If AI marks the issue as safe and self-fixable, it shows quick-fix steps with “This fixed it” / “Still need help”.	Reporter
FR-05	“This fixed it” logs a self-resolved case and creates no ticket.	Reporter
FR-06	“Still need help” creates a ticket pre-filled with AI category, priority and summary (editable).	Reporter
FR-07	Hazards (gas, sparks, exposed wires, flooding, structural damage) skip quick fix, are set to Urgent, and alert admins immediately.	System
FR-08	Ticket list supports filters (status, category, priority, location, date range), text search, sorting and pagination.	All
FR-09	Admin assigns or reassigns a ticket to a technician.	Admin
FR-10	Status workflow: OPEN → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED; CANCELLED from OPEN. Invalid transitions return 409.	Tech/Admin
FR-11	Every change is recorded in the ticket activity timeline (who, what, when).	System
FR-12	Users can comment on tickets they are allowed to see.	All
FR-13	Real-time notifications: new ticket (admins), assignment (technician), status change and comment (reporter + assignee).	System
FR-14	Reporter rates a resolved ticket (1–5) with optional feedback; rating closes the ticket.	Reporter
FR-15	Admin dashboard: open vs. resolved, average resolution time, top locations, tickets by category, AI self-resolution rate, AI accuracy.	Admin
FR-16	Admin exports filtered tickets to CSV (streamed download).	Admin
FR-17	Admin manages categories and locations (building, floor, room).	Admin
FR-18	SLA watcher flags tickets open for more than 48 hours (extra).	System

6. Non-Functional Requirements

Area	Requirement
Security	Passwords hashed (bcrypt, 12 salt rounds); JWT in httpOnly, Secure, SameSite=Strict cookie; Helmet headers; CORS limited to the client origin; rate limiting on login and AI endpoints; input validation and sanitization against XSS and NoSQL injection; AI API key only on the server.
Performance	Paginated list endpoints (default 10, max 50 per page); indexed columns for status, category, location, createdAt; database connection pooling via Prisma; AI triage times out after 15 s.
Reliability	Centralized error handler with a consistent JSON error format; AI failure falls back to the manual form; /health endpoint checks both databases.
Usability	Responsive SPA (mobile-first, since most reports are made from a phone); a ticket can be submitted in under 60 seconds.
Maintainability	Layered architecture (routes → controllers → services → data access); ES Modules; ESLint + Prettier; environment-based config.
Testability	Unit tests for business rules; integration tests with Supertest on a separate test database; AI service mocked.
Observability	Structured JSON logs with Pino (request ID, user ID, latency); log level from .env.
Portability	Runs with one command via Docker Compose (API + PostgreSQL + MongoDB + client).

7. System Architecture

7.1 High-level overview



7.2 Technology stack

Layer	Technology	Why
Front end	React 18 + Vite, React Router, Context API, Recharts	SPA framework (Objective 1); fast dev server; charts for dashboard.
Runtime	Node.js 20 LTS (ES Modules)	Event loop, non-blocking I/O, async/await (Weeks 1–2).
Web framework	Express 4/5	Routers, middleware pipeline (Week 4).
Relational DB	PostgreSQL 16 + Prisma ORM	Structured, related data; migrations; 1:N and N:M (Weeks 6–7).
Document DB	MongoDB 7 + Mongoose	Append-only activity timeline, flexible AI logs, aggregation (Weeks 7–8).
Validation	Zod	Request and AI-response schemas (Week 5).
Auth	bcrypt, jsonwebtoken, cookie-parser	Hashing, stateless JWT, cookies (Weeks 8–10).
Security	helmet, express-rate-limit, cors, sanitization	Hardening (Week 9).
Uploads	multer (disk storage) + fs streams	Photos and CSV export (Week 2).
Real time	Socket.io	Rooms, events, acknowledgments (Week 12).
AI	Vision-capable LLM API (e.g., Claude Haiku)	Photo + text triage; low cost per request.
Testing	Vitest + Supertest	Unit and HTTP integration tests (Weeks 10–11).
Ops	dotenv, Pino, Docker multi-stage, Docker Compose	Config, logging, containers (Weeks 13–14).
Docs	swagger-ui-express + OpenAPI 3	API documentation (Week 14).

7.3 Why two databases?

This is a deliberate design decision. **PostgreSQL** stores data with strong relationships and integrity rules (users, tickets, categories, locations, tags), where foreign keys and transactions matter. **MongoDB** stores the append-only activity timeline and AI triage logs: documents with variable shape that are written often, read together, and analyzed with aggregation pipelines. The ticket id links both stores.

> **Fallback:** if the professor prefers a single database, the activity and AI log tables can move to PostgreSQL (JSONB columns) without changing the API contract.

8. Data Model

8.1 PostgreSQL (Prisma schema)

```
enum Role     { REPORTER  TECHNICIAN  ADMIN }
enum Status   { OPEN  ASSIGNED  IN_PROGRESS  RESOLVED  CLOSED  CANCELLED }
enum Priority { LOW  MEDIUM  HIGH  URGENT }

model User {
  id           String   @id @default(uuid())
  name         String
  email        String   @unique
  passwordHash String
  role         Role     @default(REPORTER)
  isActive     Boolean  @default(true)
  createdAt    DateTime @default(now())
  reported     Ticket[] @relation("Reporter")      // 1:N
  assigned     Ticket[] @relation("Assignee")      // 1:N
}

model Category {
  id      Int      @id @default(autoincrement())
  name    String   @unique                        // Electrical, Plumbing, HVAC, IT/AV, ...
  tickets Ticket[]
}

model Location {
  id       Int      @id @default(autoincrement())
  building String
  floor    String?
  room     String?
  tickets  Ticket[]
  @@unique([building, floor, room])
}

model Ticket {
  id          String    @id @default(uuid())
  title       String
  description String
  photoPath   String?
  status      Status    @default(OPEN)
  priority    Priority  @default(MEDIUM)
  isHazard    Boolean   @default(false)
  aiSummary   String?
  rating      Int?                                 // 1..5
  reporterId  String
  assigneeId  String?
  categoryId  Int
  locationId  Int
  reporter    User      @relation("Reporter", fields: [reporterId], references: [id])
  assignee    User?     @relation("Assignee", fields: [assigneeId], references: [id])
  category    Category  @relation(fields: [categoryId], references: [id])
  location    Location  @relation(fields: [locationId], references: [id])
  tags        Tag[]                                // N:M (implicit join table)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  resolvedAt  DateTime?
  @@index([status]) @@index([categoryId]) @@index([locationId]) @@index([createdAt])
}

model Tag {
  id      Int      @id @default(autoincrement())
  name    String   @unique                        // recurring, after-hours, safety, ...
  tickets Ticket[]
}
```

8.2 Entity relationships

- **User 1:N Ticket** (as reporter) and **User 1:N Ticket** (as assignee).
- **Category 1:N Ticket**, **Location 1:N Ticket**.
- **Ticket N:M Tag** through Prisma's implicit join table.
- **Ticket 1:1 ActivityLog** (MongoDB, linked by `ticketId`).

8.3 MongoDB collections (Mongoose)

```
// activitylogs  — one document per ticket, events EMBEDDED (read together, append-only)
{
  ticketId: "uuid",                      // reference to PostgreSQL Ticket.id
  events: [
    { type: "CREATED" | "ASSIGNED" | "STATUS_CHANGED" | "COMMENT" | "RATED",
      actorId: "uuid", actorRole: "TECHNICIAN",
      from: "ASSIGNED", to: "IN_PROGRESS",   // for status changes
      text: "Replaced the faucet cartridge", // for comments
      at: ISODate }
  ],
  updatedAt: ISODate
}

// triagelogs  — one document per AI triage request (REFERENCED by userId / ticketId)
{
  userId: "uuid",
  description: "AC is dripping water on the desk",
  photoPath: "/uploads/2026/11/abc.webp",
  ai: { category: "HVAC", priority: "MEDIUM", summary: "...", isHazard: false,
        quickFix: ["..."], model: "claude-haiku", latencyMs: 2140 },
  outcome: "SELF_RESOLVED" | "TICKET_CREATED" | "ABANDONED",
  ticketId: "uuid" | null,
  finalCategory: "HVAC",                 // after user/admin correction → AI accuracy metric
  createdAt: ISODate
}
```

8.4 Example aggregation pipeline (dashboard)

```
// AI self-resolution rate and accuracy, last 30 days
TriageLog.aggregate([
  { $match: { createdAt: { $gte: since } } },
  { $group: {
      _id: null,
      total:        { $sum: 1 },
      selfResolved: { $sum: { $cond: [{ $eq: ["$outcome", "SELF_RESOLVED"] }, 1, 0] } },
      correct:      { $sum: { $cond: [{ $eq: ["$ai.category", "$finalCategory"] }, 1, 0] } } } },
  { $project: { _id: 0, total: 1,
      selfResolvedRate: { $divide: ["$selfResolved", "$total"] },
      aiAccuracy:       { $divide: ["$correct", "$total"] } } }
]);
```

9. REST API Design

Base URL `/api/v1`. Resources use plural nouns, HTTP verbs carry the action, and responses use standard status codes. All bodies are JSON except uploads (multipart/form-data) and CSV export (text/csv).

9.1 Endpoints

Method	Endpoint	Access	Description
POST	/auth/register	Public	Create Reporter account → 201
POST	/auth/login	Public (rate-limited)	Verify password, set JWT cookie → 200
POST	/auth/logout	Auth	Clear cookie → 204
GET	/auth/me	Auth	Current user profile
POST	/triage	Auth (rate-limited)	Photo + description → AI triage result
POST	/triage/:id/outcome	Auth (owner)	{resolved:true} logs self-fix; false creates ticket
GET	/tickets	Auth (scoped by role)	List with filters, search, sort, pagination
POST	/tickets	Auth	Create ticket (manual path) → 201
GET	/tickets/:id	Auth (owner/assignee/admin)	Ticket detail + timeline
PATCH	/tickets/:id	Owner (OPEN) / Admin	Edit title, description, category, location
PATCH	/tickets/:id/status	Assignee / Admin / Owner (cancel)	Validated status transition
PATCH	/tickets/:id/assignment	Admin	Assign/reassign technician
POST	/tickets/:id/comments	Owner / Assignee / Admin	Add comment → 201
POST	/tickets/:id/rating	Owner	Rate 1–5 after RESOLVED → closes ticket
DELETE	/tickets/:id	Admin	Delete ticket → 204
GET	/tickets/export	Admin	Streamed CSV of filtered tickets
GET/POST/PATCH/DELETE	/categories, /locations, /tags	Read: Auth · Write: Admin	Catalog CRUD
GET/POST/PATCH	/users	Admin	List, create staff, change role, deactivate
GET	/stats/overview	Admin	Dashboard metrics (Postgres + Mongo aggregation)
GET	/health	Public	Liveness + DB connectivity
GET	/api-docs	Public (dev)	Swagger UI

9.2 Query conventions (filtering, sorting, pagination)

```
GET /api/v1/tickets?status=OPEN,ASSIGNED&category=3&priority=HIGH
                   &location=7&from=2026-11-01&to=2026-11-30
                   &q=projector&sort=-createdAt,priority&page=2&limit=10

200 OK
{
  "data": [ { "id": "…", "title": "Projector not turning on", "status": "OPEN", … } ],
  "meta": { "page": 2, "limit": 10, "total": 47, "totalPages": 5 },
  "links": { "next": "/api/v1/tickets?…&page=3", "prev": "/api/v1/tickets?…&page=1" }
}
```

9.3 Status codes used

Code	When
200 / 201 / 204	Success / created / no content
400	Validation error (Zod) – includes field-level details
401	Missing or invalid/expired JWT
403	Authenticated but role or ownership not allowed
404	Resource not found (also used instead of 403 to hide other users' tickets)
409	Invalid status transition or duplicate (e.g., email already registered)
413 / 415	Upload too large / unsupported file type
429	Rate limit exceeded
500 / 502 / 504	Unexpected error / AI provider error / AI timeout (fallback to manual form)

9.4 Error response format

```
{
  "error": {
    "status": 400,
    "code": "VALIDATION_ERROR",
    "message": "Request body is invalid",
    "details": [ { "path": "description", "message": "Must be at least 10 characters" } ],
    "requestId": "c1f9…"
  }
}
```

10. Middleware Stack

Order matters: each request flows top to bottom; errors skip to the centralized handler.

#	Middleware	Type	Purpose
1	requestId + pino-http	Custom / 3rd-party	Attach request ID; structured request logging
2	helmet()	3rd-party	Secure HTTP headers
3	cors({ origin: CLIENT_URL, credentials: true })	3rd-party	Allow only the SPA origin, with cookies
4	express.json({ limit: '100kb' })	Built-in	Parse JSON bodies with a size cap
5	cookieParser()	3rd-party	Read JWT cookie
6	sanitize()	Custom	Strip $ and . keys (NoSQL injection), escape HTML in text fields (XSS)
7	rateLimit (global / login / triage)	3rd-party	100 req/15 min global; 5 login attempts/15 min; 10 triage/hour/user
8	authenticate	Custom	Verify JWT, load user, set req.user; 401 if invalid
9	authorize(...roles)	Custom	403 if req.user.role not allowed
10	validate(schema)	Custom (Zod)	Validate body/params/query; 400 with details
11	upload (multer)	3rd-party	5 MB limit, image MIME whitelist, random file names

#	Middleware	Type	Purpose
12	asyncHandler(fn)	Custom wrapper	Forward rejected promises to next(err)
13	notFound	Custom	404 for unknown routes
14	errorHandler(err, req, res, next)	Custom	Map errors (Zod, Prisma, JWT, AppError) to the JSON error format; hide stack in production

```
// src/middleware/authorize.js
export const authorize = (...roles) => (req, res, next) =>
  roles.includes(req.user.role) ? next() : next(new AppError(403, 'FORBIDDEN', 'Not allowed'));

// src/routes/tickets.routes.js
router.patch('/:id/assignment',
  authenticate, authorize('ADMIN'), validate(assignSchema), asyncHandler(ticketCtrl.assign));
```

11. Authentication, Authorization & Security

11.1 Authentication flow (stateless JWT)

- **Register:** Zod validates email/password (min 8 chars, letter + number) → bcrypt.hash(password, 12) → user saved with role REPORTER.
- **Login:** find user by email → bcrypt.compare (always run compare, even for unknown emails, using a dummy hash, to resist timing attacks) → sign JWT { sub, role } with 1 h expiry.
- **Token storage:** JWT sent in an `httpOnly; Secure; SameSite=Strict` cookie, so JavaScript cannot read it (XSS protection) and it is not sent cross-site (CSRF protection).
- **Each request:** authenticate middleware verifies signature and expiry, checks the user is still active, and sets req.user.
- **Logout:** cookie cleared. (Discussion point: JWT vs. stateful sessions — JWT chosen for stateless scaling; trade-off is no server-side revocation before expiry, mitigated by the short 1 h lifetime and the isActive check.)

11.2 Authorization

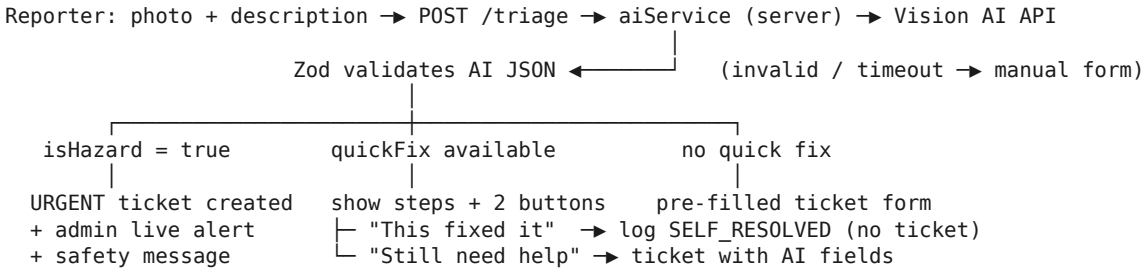
Two layers: **role guards** on routes (authorize('ADMIN')) and **ownership rules** in services (reporter sees own tickets; technician sees assigned). List queries are scoped automatically by role in the ticket service, so a reporter can never fetch other users' tickets by changing a query parameter.

11.3 Security hardening checklist

Threat	Mitigation
Brute-force login	express-rate-limit: 5 attempts / 15 min per IP + email
Password leaks	bcrypt with 12 salt rounds; passwordHash never returned (Prisma select / omit)
XSS	Escape user text on input; React escapes output by default; no dangerouslySetInnerHTML; Helmet CSP
NoSQL injection	Strip keys starting with $ or containing . ; Zod enforces primitive types
SQL injection	Prisma parameterized queries only; no raw string concatenation
Malicious uploads	MIME + extension whitelist, 5 MB limit, random file names, files served from a static folder with no execution
Secrets exposure	All keys in .env (never committed); .env.example in repo; AI key only on the server
AI cost abuse	Triage rate limit per user; 15 s timeout; max image size
Mass assignment	Zod schemas whitelist fields (e.g., a reporter cannot send role or assigneeId)

12. AI Smart Triage

12.1 User flow



12.2 AI response contract (Zod schema)

```
export const triageSchema = z.object({
  category: z.enum(['ELECTRICAL', 'PLUMBING', 'HVAC', 'IT_AV', 'FURNITURE',
                    'CLEANING', 'STRUCTURAL', 'OTHER']),
  priority: z.enum(['LOW', 'MEDIUM', 'HIGH', 'URGENT']),
  summary:  z.string().max(200),          // what the photo shows, for the technician
  isHazard: z.boolean(),
  quickFix: z.array(z.string().max(160)).max(5).nullable()  // null when not safe/possible
});
```

12.3 Prompt design (summary)

- System instruction: act as a facilities triage assistant; return **only** JSON matching the schema.
- Quick fixes must be safe for a non-technician: no tools beyond a remote or a switch, no opening panels, no ladders, no water/electric contact.
- If there is any sign of gas, smoke, sparks, exposed wiring, flooding, or structural damage: isHazard = true, priority = URGENT, quickFix = null.
- Category list and priority definitions are passed in the prompt so output matches database enums.

12.4 Safety guardrail (defense in depth)

The server does not trust the model alone. After validation, a `hazardCheck()` function scans the description and AI summary for hazard keywords (gas, smoke, spark, burning, exposed wire, flood, crack in wall/ceiling, etc., in English and Spanish). If any match, the server forces isHazard = true, priority = URGENT, and removes quick-fix steps, even if the model said otherwise. This rule is unit-tested.

Example demo cases: (1) Projector shows “No signal” → quick fix: press Source, select HDMI 1 → *This fixed it*. (2) Leaking faucet → no safe quick fix → pre-filled Plumbing/Medium ticket. (3) Exposed wires near outlet → Hazard → Urgent ticket + live admin alert.

12.5 Reliability & cost

- AI called only from the backend (key in .env). Images are resized/compressed before sending.
- 15 s timeout with AbortController; on failure the user sees the manual form (no blocked reporting).
- Every triage stored in MongoDB → feeds “AI self-resolution rate” and “AI accuracy” dashboard metrics.
- In tests, aiService is mocked so no real API calls are made.

13. Real-Time Notifications (Socket.io)

The Socket.io server shares the HTTP server. On connection, the JWT cookie is verified in a Socket.io middleware (`io.use(...)`); unauthenticated sockets are rejected. Each socket joins rooms based on identity and role.

Room	Members	Purpose
user:{userId}	Each user	Personal notifications (assignment, status change on own ticket)
role:ADMIN	All admins	New tickets, hazard alerts, SLA alerts
ticket:{ticketId}	Users viewing a ticket	Live timeline updates and comments

Event (server → client)	Payload	Sent to
ticket:created	{ id, title, priority, isHazard }	role:ADMIN
ticket:hazard	{ id, location, summary }	role:ADMIN (highlighted alert)
ticket:assigned	{ id, title, assignee }	user:{technicianId}, user:{reporterId}
ticket:status	{ id, from, to, by, at }	ticket:{id}, user:{reporterId}
ticket:comment	{ id, author, text, at }	ticket:{id}
Event (client → server)	Payload	Notes
ticket:join / ticket:leave	{ ticketId }	Server checks permission before joining; replies with acknowledgment { ok }

Events are emitted from the **service layer** after the database write succeeds, so the REST API remains the single source of truth and sockets only push notifications (with HTTP polling as the conceptual comparison discussed in Week 12).

14. Front-End (React SPA)

Page / Route	Role	Main components
/login, /register	Public	AuthForm with client-side validation
/report	All	PhotoUploader, DescriptionInput, TriageResult (QuickFixSteps, HazardBanner), TicketForm
/tickets	All (scoped)	FilterBar, SearchBox, TicketTable/Cards, Pagination (state synced to URL query)
/tickets/:id	All (scoped)	TicketHeader, StatusStepper, ActivityTimeline (live), CommentBox, RatingStars
/technician	Technician	My Queue sorted by priority, quick status buttons
/admin	Admin	Dashboard: KPI cards, charts (Recharts), unassigned tickets, export CSV
/admin/catalog, /admin/users	Admin	CRUD tables for categories, locations, tags, users

- **State:** AuthContext (current user), SocketContext (single socket connection), NotificationContext (toasts + bell counter); server data fetched per page with a small fetch wrapper (credentials: 'include').
- **Routing:** ProtectedRoute and RoleRoute components redirect unauthorized users.
- **UX:** mobile-first layout, loading and empty states, optimistic status updates with rollback on error.

15. Testing Strategy

Following the testing pyramid from Week 10: many fast unit tests, a solid integration suite, a few end-to-end checks.

Level	Tool	What is tested
Unit	Vitest	Status transition rules; hazardCheck(); helpers (pagination math, query parser); Zod schemas; AI response parsing with mocked output
Integration	Vitest + Supertest	Every protected endpoint: 401 without token, 403 with wrong role, 404 for other users' tickets, 200/201 happy paths, 400 validation, 409 invalid transition, 429 rate limit
Database	Test PostgreSQL + test MongoDB (Docker)	Prisma migrations applied to test DB; fixtures seeded in beforeEach; tables/collections truncated in afterEach (teardown)
Mocking	vi.mock()	aiService mocked to return safe, hazard, invalid-JSON and timeout cases
E2E (manual/demo)	Browser checklist	Three AI demo cases; live notification between two browser windows

```
describe('PATCH /api/v1/tickets/:id/status', () => {
  it('rejects a reporter trying to resolve a ticket (403)', async () => {
    const res = await request(app).patch(`/api/v1/tickets/${ticket.id}/status`)
      .set('Cookie', reporterCookie).send({ status: 'RESOLVED' });
    expect(res.status).toBe(403);
  });
  it('rejects an invalid transition OPEN -> RESOLVED (409)', async () => {
    const res = await request(app).patch(`/api/v1/tickets/${ticket.id}/status`)
```

```
      .set('Cookie', adminCookie).send({ status: 'RESOLVED' });
    expect(res.status).toBe(409);
  });
});
```

Target: at least 80% line coverage on services and middleware (`vitest --coverage`).

16. Configuration, Logging, Deployment & Documentation

16.1 Environment variables (.env.example)

```
NODE_ENV=development
PORT=4000
CLIENT_URL=http://localhost:5173
DATABASE_URL=postgresql://fixit:fixit@localhost:5432/fixit
MONGO_URI=mongodb://localhost:27017/fixit
JWT_SECRET=change-me
JWT_EXPIRES_IN=1h
AI_API_KEY=your-key-here
AI_MODEL=claude-haiku
AI_TIMEOUT_MS=15000
UPLOAD_DIR=./uploads
LOG_LEVEL=info
```

16.2 Logging & health

- Pino structured JSON logs (pino-pretty in development) with requestId, userId, method, path, status, latency.
- Sensitive fields (password, cookie, authorization) redacted.
- GET `/health` returns { status, uptime, postgres: 'up'|'down', mongo: 'up'|'down' } with 200 or 503.

16.3 Docker

```
# ---- build stage ----
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npx prisma generate

# ---- runtime stage ----
FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app ./
RUN npm prune --omit=dev
USER node
EXPOSE 4000
HEALTHCHECK CMD wget -qO- http://localhost:4000/api/v1/health || exit 1
CMD ["sh", "-c", "npx prisma migrate deploy && node src/server.js"]
```

docker-compose.yml runs api, client, postgres and mongo with named volumes for data and uploads. Deployment target: a cloud container host (e.g., Render, Railway or Fly.io) with managed PostgreSQL and MongoDB Atlas.

16.4 API documentation

OpenAPI 3 spec in `docs/openapi.yaml`, served with swagger-ui-express at `/api-docs`. Every endpoint documents parameters, request bodies, response schemas, error codes and the cookie security scheme.

17. Project Structure

```
fixit/
├── client/                        # React (Vite) SPA
│   └── src/
│       ├── api/                   # fetch wrapper, endpoint helpers
│       ├── context/               # AuthContext, SocketContext, NotificationContext
│       ├── components/            # FilterBar, Pagination, ActivityTimeline, ...
│       ├── pages/                 # Report, Tickets, TicketDetail, Technician, Admin
│       └── routes/                # ProtectedRoute, RoleRoute
├── server/
│   ├── prisma/                    # schema.prisma, migrations/, seed.js
│   ├── src/
│   │   ├── config/                # env.js (Zod-validated env), logger.js, db.js, mongo.js
│   │   ├── middleware/            # authenticate, authorize, validate, upload, sanitize,
│   │   │                          # rateLimiters, asyncHandler, notFound, errorHandler
│   │   ├── routes/                # auth, tickets, triage, catalog, users, stats, health
│   │   ├── controllers/           # thin: read req, call service, send res
│   │   ├── services/              # business rules: ticketService, aiService, statsService
│   │   ├── models/                # Mongoose: ActivityLog, TriageLog
│   │   ├── schemas/               # Zod schemas
│   │   ├── sockets/               # io setup, auth middleware, emitters
│   │   ├── utils/                 # AppError, queryParser, hazardCheck, csvStream
│   │   ├── app.js                 # Express app (exported for Supertest)
│   │   └── server.js              # HTTP + Socket.io bootstrap
│   ├── tests/                     # unit/, integration/, fixtures/
│   ├── docs/openapi.yaml
│   ├── Dockerfile
│   └── .env.example
├── docker-compose.yml
└── README.md                      # setup, scripts, demo accounts, architecture
```

18. Syllabus Coverage Matrix

Course objective / topic	Week	Where it appears in FixIt
Obj. 1 – Front-end framework	—	React SPA with routing, context state, live updates (§14)
Obj. 2 – V8, event loop, non-blocking I/O, ES Modules, npm	1	Node 20 ESM project; async I/O for DB, AI and file operations
Async JS: Promises, async/await, EventEmitters	1	Async services; asyncHandler; internal EventEmitter triggers socket emits
fs, path, Buffers, Streams, piping	2	Photo uploads to disk; streamed CSV export (Prisma cursor → transform → res)
Obj. 3 – HTTP request/response, status codes, body parsing	2–3	Consistent status codes (§9.3), JSON/multipart parsing
Obj. 4 – Express architecture, router module	4	One router per resource, layered controllers/services
Middleware lifecycle, custom middleware, CORS	4	14-step stack (§10)
Centralized error handling, async wrappers	4	errorHandler + asyncHandler + AppError
Obj. 5 – REST constraints, naming, HTTP verbs	5	Resource-oriented /api/v1 design (§9)
Validation & sanitization (Zod/Joi)	5	Zod for requests, env vars and AI output
Filtering, pagination, sorting	6	Ticket list query conventions (§9.2)
Obj. 8 – PostgreSQL, pooling, Prisma migrations	6–7	Prisma schema, migrations, seed (§8.1)
CRUD, 1:N and N:M relations	7	User–Ticket, Category–Ticket, Ticket–Tag
MongoDB, Mongoose schemas, embedded vs. referenced	7–8	ActivityLog (embedded), TriageLog (referenced) (§8.3)
Aggregation pipelines	8	Dashboard metrics (§8.4)
Obj. 6 – bcrypt, salt rounds, timing attacks	8	Registration/login flow (§11.1)
Obj. 7 – JWT vs. sessions, cookies	9	JWT in httpOnly cookie; trade-off discussion
RBAC, authorization guards	9–10	authorize() + ownership rules (§4.1, §11.2)
Rate limiting, Helmet, XSS / NoSQL injection	9	Security checklist (§11.3)
Testing pyramid, Vitest, Supertest	10	Unit + integration suites (§15)
Test DBs, mocking, fixtures, teardown	11	Docker test DBs, vi.mock(aiService), seed/truncate
WebSockets vs. polling, Socket.io rooms, acks	12	Rooms, events, acknowledgments (§13)
.env, structured logging, health checks	13	Pino, /health, validated env (§16)
Multi-stage Docker, cloud deployment	13–14	Dockerfile + Compose + cloud host (§16.3)
OpenAPI / Swagger	14	/api-docs (§16.4)

19. Development Timeline

Week	Dates	Milestones
1	Nov 9 – Nov 15	Repo + Docker Compose (Postgres, Mongo); Prisma schema + migrations + seed; Express skeleton, middleware stack, error handling; auth (register/login/JWT cookie) + RBAC guards; first integration tests
2	Nov 16 – Nov 22	Ticket CRUD, status workflow, assignment, comments, activity timeline (Mongo); filtering/sorting/pagination; photo upload; Socket.io rooms + events; React: auth, ticket list, ticket detail
3	Nov 23 – Nov 29	AI Smart Triage (service, Zod contract, hazard guardrail, fallback); React report flow; admin dashboard + aggregation; CSV export; Pino + /health; expand test suite (mocks, fixtures)
4	Nov 30 – Dec 6	Security review (rate limits, sanitization, Helmet); Swagger docs; multi-stage Dockerfile + deployment; README; bug fixing; peer code review (Dec 4); rehearse demo
Final	Dec 7	Final presentation and architecture review

20. Presentation & Demo Plan (≈ 10 minutes)

Time	Segment
1 min	Problem: lost WhatsApp requests, no tracking, no data.
1 min	Solution overview + architecture diagram + why two databases.
3 min	Live demo – AI triage: (1) projector quick fix → “This fixed it”; (2) leaking faucet → pre-filled ticket; (3) exposed wires → Urgent + live admin alert.
2 min	Live demo – workflow in two browser windows: admin assigns → technician updates → reporter sees live status → rating closes ticket → dashboard updates.
1 min	Security & RBAC: show 401/403 in Swagger; rate limit 429.
1 min	Quality: run test suite + coverage; Docker deployment URL.
1 min	Results, lessons learned, future work; Q&A.

21. Deliverables Checklist

- ☐ Source code repository (client + server) with clear commit history
- ☐ README: problem, features, setup steps, environment variables, scripts, demo accounts
- ☐ .env.example (no real secrets committed)
- ☐ Prisma schema, migrations and seed data
- ☐ OpenAPI spec + Swagger UI at /api-docs
- ☐ Test suite (unit + integration) with coverage report
- ☐ Dockerfile (multi-stage) + docker-compose.yml
- ☐ Deployed URL (or local Docker demo)
- ☐ Presentation slides + architecture diagram
- ☐ This specification document

22. Risks & Mitigations

Risk	Mitigation
AI gives a wrong or unsafe suggestion	Strict prompt, Zod validation, server-side hazardCheck override, user can always request help
AI API unavailable or out of credits during demo	Timeout + manual form fallback; DEMO_MODE flag returns recorded responses for the three demo cases
Scope too large for 4 weeks	MVP first (§3.2); extras only after core endpoints are tested
Two databases add complexity	Mongo used only for timeline and AI logs; documented fallback to PostgreSQL JSONB
Deployment issues	Docker Compose works locally first; deploy in Week 4 with time buffer

23. Future Work

- Push/e-mail notifications and a PWA for offline reporting.
- QR codes in each room that open the report form with the location pre-filled.
- Predictive maintenance: detect recurring failures per location from historical data.
- Spare-parts inventory and cost tracking per ticket.
- Multi-organization support (several campuses or buildings with separate admins).